

Test Planning Document

Contents

Scope	1
Goal	1
Tools	1
Instrumentation	2
Scaffolding	2
Activities and Task Schedule	2
Vulnerabilities	3

Scope

- System to test: Medical drone delivery system
- Requirements to test:
 1. Drones must not enter no-fly zones (NFZ) and restricted zones.
 2. Total time taken to calculate drone path must not exceed 200 milliseconds.

Goals

Requirement (i): All drone paths must avoid entering or touching NFZ borders. For each test case with a valid solution, the calculated path must fully avoid all NFZs.

Pass/fail criterion: path does not intersect any NFZ polygon.

Goal: 100% pass rate for solvable scenarios across varying NFZ configurations.

Requirement (ii): Path calculation must complete within 200 milliseconds. For each test case, execution time is measured from `PathFindingService.findPath()` function call to its return.

Pass/fail criterion: execution time $\leq$ 200 milliseconds.

Average runtime calculated across multiple test runs with varying NFZ database sizes (1-100 zones).

Goal: 90% of test executions complete within the 200 milliseconds limit.

Aim to achieve 95% statement coverage and 85% branch coverage.

Tools

- JUnit - test framework
- IntelliJ IDEA's code coverage tool (JaCoCo) - code coverage analysis
- Pit - mutation testing

Instrumentation

- `System.currentTimeMillis()` - for recording run time
- Assert statements - correctness validation
- `@BeforeEach` - test data initialization
- Console logging - track test results

`System.currentTimeMillis()` is able to record time in milliseconds which should be enough precision for the testing. However, the tests will not be able to identify any bottlenecks within `findPath()` when they arise as the runtime is calculated from the start of `findPath()` call to its return. The tests can be further improved by recording the time taken for different sections within `findPath()`.

Scaffolding

- Random NFZ generator - generates triangular areas given an input database size
- Predefined NFZs - unique edge cases for extreme scenarios
- Fixed coordinate pairs - 2 pairs, a close and a far distance

The scaffolding allows for adequate testing however there may be several limitations which are discussed in the *Vulnerabilities* section.

Activities and Task Schedule

Testing approach & lifecycle: the testing will use a post-development regression testing approach. The pathfinding algorithm was developed prior to this testing work so these tests will validate the existing implementation and are meant for integration into a CI/CD pipeline for continuous regression testing for any future changes.

1. Build the test infrastructure
 - NFZ generator - for an input integer n , outputs n number of randomly generated triangular NFZs in Edinburgh
 - Define origin-destination pairs, one pair each for a close distance and a far distance
 - Create predefined irregular NFZs for edge cases
 - Timing instrumentation
2. Unit testing for the calculation helper methods used in `PathFindingService`
 - Dependency: None
3. Test `findPath()` correctness against irregular NFZs
 - For each case, execute tests twice (once for the close distance and another for the far distance).
 - Dependency: predefined origin-destination pairs and irregular NFZs from step 1
4. Test `findPath()` correctness against varying NFZs
 - For each NFZ database size (0, 10,..., 100), run the test 50 times for the close distance and 50 times for the far distance.
 - Expect to run a total 1002 tests here.
 - All tests should pass and not intersect with any NFZs.
 - Dependency: NFZ generator and origin-destination pairs in step 1
5. Test `findPath()` performance against varying sizes of NFZ databases
 - For each NFZ database size (0, 10,..., 100), run the test 50 times for the close distance and 50 times for the far distance.
 - Record the runtime of each test and compute the average runtime for each NFZ size and distance combination.
 - Expect to run a total 1002 tests here.
 - Dependency: Functional correctness of `findPath()` must be verified in step 3 and 4

6. Evaluate the test suite using mutation testing
 - Dependency: Test suite from step 3-5 must be completed

Vulnerabilities

- Test suite will not contain a database of > 100 NFZ due to limited computational power and time constraints
 - Given that the drone delivery system is only operational within Edinburgh, 100 NFZ is adequate. According to UK NATS UAS Restriction Zones, there is only 2 restricted areas within Edinburgh [1]. Additional restricted areas can be set up by the CAA during certain festivals [2] but 100 NFZ is more than adequate for Edinburgh alone.
- Tests will not be using all possible origin-destination pairs and will only use 2 pairs (one short distance, another long distance).
 - The two pairs covers different path complexities (close and far) and with up to 100 varying NFZ, the drone paths will vary enough to make this limitation minor.
- All NFZ are triangular.
 - Based on the existing NFZ from NAT, these areas are mostly irregular polygons or circular areas. Irregular polygons are not considered for the NFZ generator due to its complexity.
 - An alternative considered is to use rectangles, however after careful assessment, triangles better mirror irregular shapes than rectangles can thus triangular NFZ are chosen.
 - If the task schedule is ahead of time, extend the NFZ generator to generate irregular polygons to counter this vulnerability.

References

- [1] NATS, “Uas restriction zones,” 2025, Retrieved on 31 December 2025. [Online]. Available: <https://nats-uk.ead-it.com/cms-nats/opencms/en/uas-restriction-zones/>.
- [2] I. Storie, “Edinburgh city centre no-fly restrictions in place during hogmanay events,” 2025, Retrieved on 31 December 2025. [Online]. Available: <https://www.edinburghlive.co.uk/news/edinburgh-news/edinburgh-city-centre-no-fly-33134208>.